Translating Boundaries into Code: System Architecture of an AI Support Agent for Neurodivergent Developers

Michael Liu, Adrian Zhuang,
Mentor: Ren Butler Advisor: Prof. Andrew Begel

Research Question

How can we architect a stateful AI system in web-based IDE that synchronizes frontend user controls with background monitoring and preserves context across system restarts?

System Requirements

Asynchronous Compiling Event Monitoring. Passively intercept Jupyter kernel execution errors without blocking the main thread.

Real-Time State Synchronization. Uses local configuration files to sync frontend UI toggles with the backend error watcher.

Persistent Context Memory. Serializes chat history to disk to maintain continuous context across reloads.

Engineering Objective

Architect a stateful backend engine and a responsive frontend UI integrated within web-based computational notebook to execute proactive, boundary-aware support.

Tech Stack

Frontend Environment: JupyterLab, React, ui-components library by JupyterLab

Backend Infrastructure: Jupyter Server Extension, Python

Agent Persona: LLM-powered persona with disk-based session persistence

Backend Architecture

Jupyter Kernel (Active code)

Monitors execution events
Listens to ZMQ *iopub* sockets for *execute_reply* & *error* payloads

Asynchronous Error Watcher

Detect code error patterns
Routes threshold triggers via REST API to local config.json file

Shared Config File System

When "Active Mode" is enabled
Aborts AI check-in when JSON state is false (user turn off active mode)

Persona Engine ⟷ **Maintains context persistence** ⟷ **Disk-Based Session State**

State machine orchestrating distinct AI persona prompts

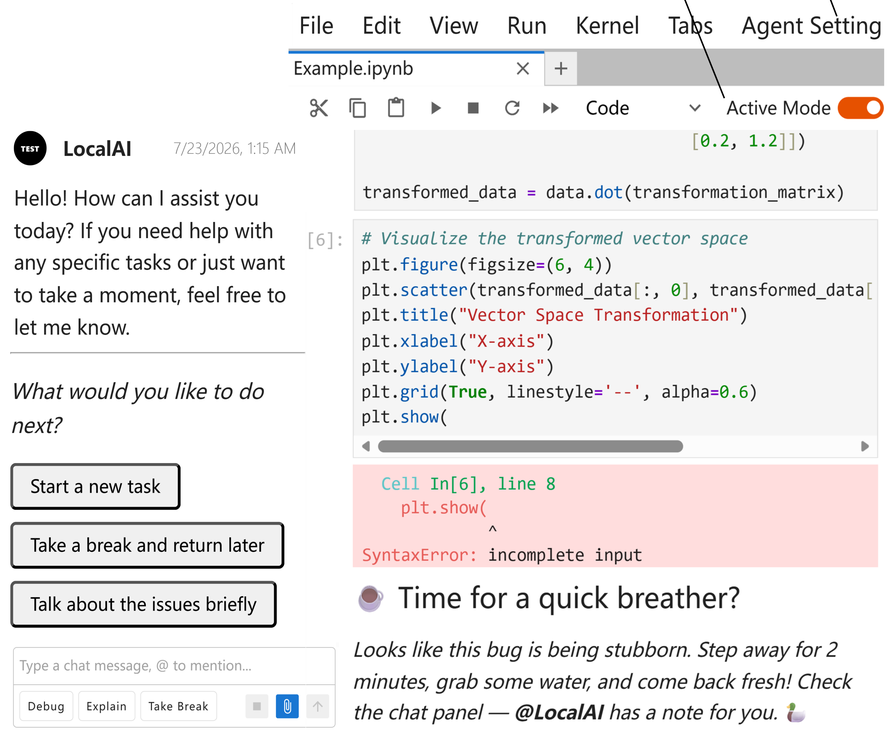
Serializes chat history to local JSON files for session recovery

Generate AI check-in

Frontend Effect

React Custom UI allows tuning of the agent's persona, e.g. TA, friend (Through *ReactWidget* class by *apputils* package)

Configuration Toggle allows user override which disables AI check-in (Through *IWidgetExtension* interface)



Engineering Solutions

State Synchronization: Engineered REST APIs to route UI toggles to local config.json, enforcing user boundaries.

Session Persistence: Configured Jupyter Server entry points for the React frontend to recover serialized context across system restarts.